Estructura de Datos y Algoritmos

Jaime Nazar Anchorena

Contents

Unidad 1: Algoritmos **2**
- Tiempo de ejecucion . . . 2
  - Empiricamente . . . 2
  - Teoricamente . . . 2
  - Calculo del Big O . . . 2
- Maven . . . 3
  - Goals . . . 3
  - Build Phases . . . 3
  - `pom.xml` . . . 4
  - Como correr el `.jar` fuera del entorno . . . 4
- Test-Driven Development . . . 4
  - Unit Testing . . . 5
  - Como testear en un Unit Test . . . 5
  - JUnit 5 . . . 5
  - Comparando resultados . . . 6
  - Ambiente de prueba . . . 6
  - JUnit Coverage . . . 7

Unidad 1: Algoritmos

El analizis de algoritmos nos permite caracterizar la cantidad de recursos computacionales el usara un algoritmo dado, estas son:

1. Tiempo de ejecucion
2. Espacio que utilizan

Tiempo de ejecucion

Como los mido? Hay dos caracteristicas, ambas sirven, pero apuntan a cosas diferentes

1. Empiricamente
2. Teoricamente

Empiricamente

Debe ocurrir en la misma maquina, se puede haver con las clases `Timer` que hicimos en clase.

> **NOTA:** Cuidado, a veces un algorimo parece mejor que otro, pero en casos extremos puede ser peor o incluso fallar.

Teoricamente

Da una tendencia, no un tiempo dado. Categoriza los algoritmos, por ejemplo, todos los O(n^2). Consiste en una descripcion de alto nivel del algoritmo para evaluar su eficiencia, idependientemente del hardware y el software donde se ejecuto.

Para contar las operaciones y encontrar Big O, hay que contar las operaciones fijas y los ciclos. Hay que tener en cuenta el peor de los casos y encontrar la menor O grande. Si aparece una llamada a una funcion, hay que revisar la documentacion para verificar el numero de operaciones que requiere.

Calculo del Big O

La descripcion que buscamos para comparar algoritmos es una asintota(cota). Comportamiento asintotico superior o Big O se define como:

Sean T(N) y g(N) funciones con N > 0

Se dice que T(n) es O(g(N)) si Existe c > 0(no depende de N) y Existe n_c > 0 tal que Para Todo N >= n se cumple que 0 <= T(N) <= c * g(N). Ejemplo completo del campus:

Definición de Comportamiento asintótico superior u O grande (*asymptotic upper bound running time u O-notation*) de un algoritmo.

Sean T(N) y g(N) funciones con N >0.
Se dice que T(n) es O (g(N)) sii $\exists$ c > 0 (constante no dependiente de N) y $\exists$ n_0 > 0 tal que$\forall$ N $\geq$ n_0 se cumple que 0 $\leq$ T(N) $\leq$ c * g(N).

Ej: En nuestro caso, algoA tiene T(N) = 3 * N
Pero para hacer más complicada la discusión, supongamos que T(N) = 1 + 3 * N, o sea g(N) puede ser N.
O sea, si 0$\leq$ 1+3*N $\leq$ c * N ¿cuánto debe valer c?

0$\leq$ 1/N + 3 $\leq$ c
Si N es 2 entonces c >= 3.5
Si N es 5 entonces c >= 3.2, mejor la cota.
Si N es 500, c >= 3.002, mejor aún.
Yo quiero N$\rightarrow\infty$, entonces c >= 3. Así, el orden de algoA es O(N).

Encontré caracterizar una constante c, que es 3, para el cual ser verifica la fórmula para N$\rightarrow\infty$ ($\forall$ N $\geq$ n_0). El algoritmo es O(N).

Maven

Maven es un manejador de proyectos en Java, como objetivos propone:

- Proporcionar un sistema de construccion uniforme
- Proporcionar informacion de calidad del proyecto
- Proporcionar pautas para el desarrollo de mejores precticas.
- Permitir la migracion transparente a nuevas funcionalidades.

Como su funcionalidad principal, permite declarar **dependencias** para utilizar librerias externas o nuestras. Maven se encarga de compilar el proyecto, armar el `.jar` con lo que ya compile, y corre los test. Tambien, guarda los projectos localmente para poder utilizar dentro de otros proyectos.

Maven funciona de manera algoritmica, primero busca localmente la dependencia en nuestro repositorio local. Caso contrario, lo busca en internet, si lo encuentra, lo descarga.

Goals

Tareas especificas dentro del build, por ejemplo:

- `mvn jar.jar`: Arma un `/jar` desde codigo ya compilado
- `mvn dependency:tree`: Muestra las dependencias
- `mvn exec java`: Corre el proyecto, `exec` es el plugin y `java` es el goal.

Build Phases

Etapas del armado del proyecto, por ejemplo: - `mvn compile` - `mvn test`

pom.xml

Aca declaro como sera my libreria, como aparecera para los demas en el repositorio. Ademas, se le pueden agregar dependencias que tu codigo utilize.

```
<groupId>ar.edu.itba.eda</groupId>
<artifactId>TimerJoda</artifactId>
<version>2</version>
```

Donde `ar.edu.itba.eda` no es el nombre del paquete donde se encuentra el codigo, sino que es el directorio donde se guarda la libreria en el repositorio.

Como correr el `.jar` fuera del entorno

Fuera del entorno de desarrollo, tenemos que poder correr nuestro programa(como el cliente). Para poder compilar todo el entorno en el archivo, hay que agregar un plugin en el `pom.xml`. Lo que va a hacer es juntar todas las dependencias en un unico `.jar`, es basicamente instrucciones extra. Logicamente, el ejecutable generado ocupara mucho mas espacio.

Se puede ejecutar especificando donde esta la clase main.

```
java -cp ***.jar core.AnalisysTimer
```

Pero tambien es posible simplemente decirle que es un ejecutable, pero para que sepa donde esta el main hay que ponerlo en el manifest(independiente de maven), sino da error. Ademas, maven tiene la opcion de agregar en la seccion de plugin para que haga eso automaticamente.

```
<configuration>

    ...

    <archive>
        <manifest>
            <mainClass>core.AnalisysTimer</mainClass>
        </manifest>
    </archive>

    ...

</configuration>
```

Luego, puedo correr my programa con:

```
java -jar ***.jar
```

Test-Driven Development

- TDD es una metodologia que comienza por los test y luego pasa a la implementacion.

- Permite enfocarse en la definicion de la interfaz y del comportabmiento y no en los detalles de implementacion.
- Ve al sistema que se va a desarrollar como una caja negra que todavia no existe.
- Apunta a que todas las funcionalidades tengan algun test.

Unit Testing

Para testear pertes individuales del programa.

- Automatico
- Verifican un unico caso por test
- Repetible
- Independientes de otros test o de condiciones externas.

Como testear en un Unit Test

- Caso de metodo o funcion:
  - Casos tipicos
  - Casos de borde
  - Casos de error
  - Casos de excepcion
- Caso de una clase:
  - Secuencias de llamadas validas
  - Secuencias de llamadas invalidas
  - Chequeo de invariantes

JUnit 5

Framework de java para realizar pruebas de desarrollo de una aplicacion, ha muchos, pero vamos a usar este. Puede comparar resultados de las invocaciones de metodos, o verificar si una excepcion fue lanzada o no. Ademas, un caso de prueba es abortado ni bien falla alguna verificacion.

Para que funcione con maven y poder hacer `mvn test`, hay que agregar un plugin:

```
<plugin>
    <groupId>org.apache.maven.plugins</groupId>
    <artifactId>maven-surefire-plugin</artifactId>
    <version>3.5.2</version>
</plugin>
```

Y en dependencias:

```
<dependency>
    <groupId>org.junit.jupiter</groupId>
    <artifactId>junit-jupiter-engine</artifactId>
    <version>5.12.0</version>
```

```
    <scope>test</scope>
</dependency>
```

Comparando resultados

El test unitario mas simple consiste en comparar el resultado obtenido con el resultado esperado, para ello se puedde utilizar metodos estaticos como `asserEquals(valorEsperado, valor Obtenido)` o `assertTrue()`.

Ademas, si un metodo lanza una excepcion, el mismo se considera que fallo. Si se espera que lanze una excepcion, si indica como `assertThrows(RuntimeException.class, () -> ...)`. Tambien puede teatear y comparar valores de punto flotante, basicamente tiene funciones para casi todos los casos de pruebas.

Para Maven, en las clases con los test, hay que agregar la palabra clave `Test` en el nombre de la clase, de esa manera Maven va a carrer automaticamente los test en el deployment. Tambien, es obligatorio agregar un `@test`.

```
class TimerTest {

    @test
    @DisplayName("Probar el timer") <- opcional
    void getDurationExceptionTest() {
        ....

        assert...();
    }
}
```

> **NOTA:** Testear Strings es muy dificil(no recomendado), fallan por la mas minima diferencia.

Ambiente de prueba

Frecuentemente se desea tener un ambiente de prueba perfijado(no tener que declarar la misma variable todo el tiempo), para eso se usan anotaciones especiales:

- `@BeforeAll` - Se ejecuta entes de todos los casos de prueba
- `@BeforeEach` - Se ejecuta antes de cada `@Test`
- `@AfterEach`
- `@AfterAll`
- y mas.

Ejemplo:

```
@BeforeAll
static void initAll() {
    ...
}
```

```
@AfterEach
void tearDown() {
    ...
}
```

Teniendo en cuenta `MyTimer`, podriamos hacer:

```
private MyTimer timer;

@BeforeEach
void instanceTimer() {
    this.timer = new MyTimer();
}
```

JUnit Coverage

Como verifico que el testeo recorra todo el codigo?

JUnit contempla eso, si quiero ver la covertura de mis test, JUnit generara un reporte.